BREATHAPP · GUARDRAILS · NOVEMBER 2026

The route of one change

How 42 guardrails protect the code of a breathing app from the mistakes of agents and people. What they catch, why they exist, and where they still leave a gap.



28 work automatically

7 partial

7 still missing

From the siroroman/BreathApp repo and its GitHub settings

Status as of 1 Nov 2026

INTRODUCTION

A fast colleague with no fear

Breathnest is a small breathing app. AI agents write much of its code. This is the story of how we keep them on the road.

At first glance Breathnest is simple. You pick an exercise, say box breathing, set a length, and a circle on the screen shows you when to breathe in and when to breathe out. Underneath sits an engine that has to time every Phase exactly, a Log that must never lose a single Session, and reminders that must still ring a week from now. The app is offline and keeps everything on the phone. It has no backup.

Agents write a large part of this code. The *implementer* takes a ticket and carries it all the way to a pull request. The *code-reviewer* reads it in a fresh session with a clear head. The *ui-verifier* starts the simulator and looks at the screens. The *improver* turns observations from the work into process changes every day.

In many ways an agent is an ideal colleague. It does not get tired, it does not skip tests out of laziness, and in an hour it does an afternoon of work. But it has two traits that would worry you in a person. It has no fear and it has no memory. It does not know that something went wrong last week, and when something blocks it, it reaches for the shortest path it knows. Sometimes that is `git reset --hard`.

So it is not enough to write the rules in a document. They have to be built like a guardrail on a mountain road. While you drive along the road you do not even know it is there. It only matters when you start to slide off. Such barriers are called *guardrails*.

Five slices of cheese

Aviation and hospitals think about safety through a simple picture. Every protective layer is a slice of Swiss cheese with holes. No slice is perfect. An accident happens only when the holes in all the slices line up. BreathApp has five such slices, and they follow exactly the road every change travels: from the agent's session, through the disk and the pull request, to the gate into `main` and the repo settings.



We will walk this road step by step. At each stop you will see what is guarded there, which problem it solves and where the gap is. The stories in the *It happened* boxes are real cases from the history of the repo, with the number of the pull request in which they took place. The details of all 42 guardrails are in the interactive map `docs/guardrails-map.html`.



FIRST STOP

While the agent works

5 in place 2 partial

The first layer lives right inside the Claude Code session on the Mac. It acts before a command even runs, which makes it the cheapest. A mistake that does not happen costs nothing.

The base is a short list of allowed commands. The agent may run builds, tests, lint, the simulator and read-only git commands without asking. Everything else stops and waits for you to approve it. That way the agent does not nag you on every `make test`, and at the same time it cannot install a package or delete a folder without your knowledge.

Then there is a list of things that cannot be approved at all. The agent may not read `.env`, keys, certificates or `~/.ssh`. It may not run `curl`, `wget`, `sudo` or anything that ends in `| sh`. That last ban has a prosaic reason. Many guides on the internet tell you to install a tool with a single line that downloads a script and runs it at once. An agent that finds such a guide would obey it willingly. Since 1 November `gh pr merge`, `gh api` and `gh repo delete` are denied as well. Merging stays a human decision.

IT HAPPENED PR 6

A hook that first blocked everything and then too little

The most important guard of the first layer is the hook against dangerous git. Its first form came from a downloaded skill and blocked every push, so the agent could not even open a pull request. The rewritten version had the opposite problem. A review found bypasses in it: it was enough to write `git -C dir push -f` or `-fu` instead of `-f` and the hook let the command through. A test script was added that showed 36 failures on that version. Today it runs in CI on every pull request, so the same hole cannot quietly come back.

Today the hook stops force push, push to `main`, hard reset, forced deletion of branches and files, and the command that throws away all unsaved changes. It lets a push of the agent's own branch through, so the agent can open a pull request. When the tool `jq`, which the hook uses to read the command, is missing, it prefers to refuse everything rather than let something through blindly.

A less dramatic but useful hook runs swiftformat after every edit of a Swift file. The agent then does not waste time fixing spaces and brackets that would otherwise fail lint.

The last help of the first layer arrives at the end. When the agent wants to finish and has changed Swift or project files, the Stop hook runs `make lint test`. If something fails, it hands the agent the last 60 lines of the error, and the agent carries on and fixes it instead of announcing it is done.

A HOLE IN THE CHEESE

By the rules the agent commits after every small step. At the end of a session the working tree is therefore clean, and a Stop hook that looks only at uncommitted changes has nothing to check. A second attempt to stop is always let through, so the agent does not loop. The fix is described in issue #63. The remaining working rules, for example when a task counts as done, exist only as text in `CLAUDE.md` and `docs/agents/` for now.



SECOND STOP

Commit and push on disk

1 in place | 1 partial | 2 missing

When an agent or a person makes a commit, the git hooks in the `.githooks/` folder take over the watch. There are two and both are fast.

The first checks the commit message. The project uses Conventional Commits, so every message starts with a type of change. A message like `fix stuff` says nothing about the change. A message like `fix(session): keep the Lead-in out of the length` says what changed, where, and what kind of change it is. Such a history can be read and filtered, and a changelog can be built from it.

IT HAPPENED the first commit

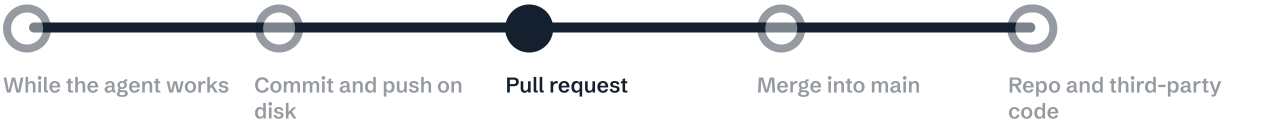
A monument to the time before the rules

The very first commit in the repo is called `Initial commit: .gitignore`. It does not meet the convention, because it was made before the hook existed. Nobody fixed it, and on purpose: rewriting history that is already on GitHub is worse than one ugly message. It stays there as a monument to the time before the rules. Today the hook would reject it and show an example of the right shape.

The second hook runs the quick Kit tests before every push, that is, the logic of the app without the screens. They take seconds and need no simulator. A mistake that would otherwise surface in CI ten minutes later, when you are already doing something else, shows up at once, while you still have the code in your head.

THE WEAKEST PLACE ON THE WHOLE ROUTE

The hooks are switched on by hand with one command in every clone, and whoever forgets it does not have them. The `--no-verify` switch skips them. Git on disk happily accepts a commit straight on `main`, and only GitHub saves you, because it rejects the push. Locally nobody looks for keys in commits either, so a key of an unknown format can get all the way to GitHub. Issue #66 deals with the first three of these, and it is the first item on the list of what is left.



THIRD STOP

Pull request

11 in place · 3 partial · 3 missing

The pull request is where the most guards meet. CI on GitHub runs six required checks, and each one watches something different.

`Script tests`	Tests of the guards themselves: the hook against dangerous git and the script that prepares the facts for a PR.
`Kit tests`	The logic in `BreathnestKit` on macOS: Session, Phases, timing.
`Build, app tests, lint`	The whole app, its tests and the code format with a pinned swiftformat version.
`check-title`	The PR title in the Conventional Commits shape, because with squash merge it becomes the commit on `main`.
`check-body`	The six sections of the PR body, and a Guardrails touched line when the PR changes a guard.
`critical-paths`	A test with every change in a sensitive part of the app.

IT HAPPENED PR 96 and PR 98

A green that lied

The most instructive story of the project took place on 1 November. An agent was working on making every compiler warning an error. It checked its work honestly: it added a line with an unused variable to a file and expected the build to fail. The compiler really did print an error. But `make build` ended with code 0, as if nothing had happened.

The cause was buried deep. macOS still ships GNU Make 3.81 from 2006, and it ignores the `.SHELLFLAGS` setting, where `pipefail` had been placed. The output of `xcodebuild` flowed through a pipe into `xcbeautify`, and make saw only the result of `xcbeautify`. That was always a success. The job that was supposed to guard the build and the app tests could therefore not go red, neither on a broken build nor on a failing test. All along it looked like a guard and guarded nothing.

The agent wrote this into the notes for the reviewer, issue #97 was created, and the fix followed the same day: one line in the `Makefile` and a test that checks, on macOS, that a failure inside a pipe fails make.

A check that cannot fail is worse than no check. At least no check fools nobody.

Who guards the guards

From this follows a question that the project takes seriously: what if the check itself breaks? There are two answers. First: every check script has its own test, and that test runs in CI right before the check. If somebody edits the hook and it stops catching a force push, the test goes

red. Second: a change to a guard has to be declared. When a pull request edits a hook, a workflow, a check script, the `Makefile` or the list of guarded paths, its body must contain a Guardrails touched line saying what got stricter and what got looser. Agents edit guards too, above all the daily improver.

IT HAPPENED PR 89

A list that could be emptied

While this check was being built, a test uncovered a sneaky hole. The list of guarded paths is in the file `guardrail-paths.md`, and the script read it from the branch of the pull request. A PR that emptied the list therefore passed the check. After all, by an empty list nothing guarded had changed. The fix: the script reads the list from `main` too, so deleting a path no longer hides its own change.

The PR body and the sensitive parts of the app

The body of every pull request has six sections: Summary, Tests, Snapshots, Evidence, Merge Danger and Reviewer notes. This is not bureaucracy. A reviewer, whether a person or an agent, needs to know what was tested, what can break and what the author did not verify. Before PR 81 even a pull request with an empty body passed. Today `check-body` stops it and says which section is missing.

Critical paths are the parts of the app where a mistake hurts the most: the Session engine and timing, the identity of exercises, haptics, running in the background and stored data. When the ID of an exercise changes, old records in the Log and the reminders quietly start to mean something else. When the shape of stored data changes without a migration, the user loses their history and has no backup. A pull request that changes code in such a part and changes no test is therefore stopped by `critical-paths`. If a test truly cannot be written, the author starts the PR body with a warning and the reviewer sees it first.

The other guards are quieter. Since PR 96 every compiler warning is an error, so warnings do not pile up until the important one is lost among a hundred. CI installs exactly swiftformat version 0.51.13, because the newer Homebrew version had new rules and reported errors in files nobody had touched.

PARTIAL AND MISSING

The review is done by the code-reviewer agent in a fresh session, and it can be valuable: it was a review that found the bypasses in the first hook. But it is started by hand, and nothing enforces that it happened. CI takes a screenshot of the app in the simulator, but nobody compares that picture with a reference. The script `pr-facts.sh` prepares the facts for the PR body, yet it is a helper, not a check.

And there are three holes. We do not measure how much code the tests cover. No automated UI tests check the screens, not even at the largest text size AX5, where an exercise name easily wraps across the Start button. And no linter reports a break of the conventions: when the agent uses `Date()` in the Kit instead of the injected `Clock`, it passes build and lint, although by the rules of the project it is a mistake. For the last two, see issues #72 and #64.



FOURTH STOP

Merge into main

4 in place · 1 partial · 1 missing

The fourth stop is the gate, and it is the firmest. Whatever slipped through the holes of the earlier layers stops here.

Since 1 November a ruleset on `main` lets nobody push directly, not even an admin. It forbids force push and deleting the branch. A merge is possible only through a pull request with six green checks and a branch that is up to date with `main`. An admin can bypass the rule only knowingly, when merging a pull request. Without this rule CI would be just advice, and the Merge button could be pressed over a red build.

The repo allows only squash merge. Every pull request is one commit on `main` with the PR title. Intermediate steps like `wip` or `fix typo` stay on the branch, the history reads as a list of finished things, and undoing a change means one revert. After the merge the branch is deleted on GitHub by itself, so nobody can carry on by mistake on a branch that is already long in `main`.

A HOLE THAT WILL START TO MATTER LATER

The ruleset requires zero approvals, because GitHub does not let an author approve their own pull request and one person works on the project. There is also no CODEOWNERS file that would say who must see a change in the Session engine or in the Log. Once there is a second person on the team, both will be due.



FIFTH STOP

Repo and third-party code

7 in place 1 missing

The last layer guards the surroundings: keys and tokens, third-party tools, and CI itself.

Secret scanning searches the repo and warns about a key that has got into it. Push protection goes a step further and does not let a push with a key of a known format through at all. A key that once leaks into git history cannot simply be deleted by the next commit. It stays in the history and has to be revoked and replaced.

Actions in CI, for example `actions/checkout`, are third-party code that runs with access to the repo. That is why they are pinned to a specific commit, not to a tag. The tag `v7` can be moved to different code by the author of the action, and just as well by somebody who steals their account. The fingerprint of a commit cannot be moved. The token that actions receive can only read the repo, so even a tampered action could not write to it.

IT HAPPENED PR 78

Dependabot keeps versions fresh

Pinned versions have to be updated, or they get old and miss security fixes. Nobody would follow that by hand, so Dependabot does it. On 30 October it opened a pull request by itself that moved `actions/checkout` from four to seven. All that was needed was to check it and merge it. It is ready in the same way for the Swift package `BreathnestKit`. The package has no dependency yet, so Dependabot is waiting for the first one.

Every job in CI has had a time limit since PR 99. Without one GitHub would let a stuck job run for six hours and block the pull request. The slowest run so far took about nine minutes, and the limit is thirty.

A HOLE

Security analysis of the code is missing, for example CodeQL. For an offline app with no accounts the risk is small today. It will start to matter once there is more code and the Log starts to store more data.

WHAT NEXT

What is left

The holes from the earlier chapters, ordered by how much risk they close for little work.

1 **Stop commits on main and bypassing the hooks** issue #66

A `pre-commit` hook that refuses a commit on `main`, a ban on `--no-verify` in the Claude hook, and a `make setup` that switches the hooks on by itself.

2 **Run the Stop hook after a commit too** issue #63

The hook should compare the whole branch with `main`, not only the uncommitted changes.

3 **Swift conventions in make lint** issue #64

SwiftLint with the project rules, for example no `Date()` in Kit logic, and a script that guards that the Kit does not import UI frameworks.

4 **The same tool versions everywhere** issue #71

A `Brewfile` and a check of the swiftformat and SwiftLint versions, so local lint gives the same result as CI.

5 **UI tests and an accessibility audit from the first screen** issue #72

A UI test target with an accessibility audit and a translations check, while there are few screens and adding them is cheap.

6 **Later: coverage, CodeQL, CODEOWNERS, a local secret scan**

They make sense once the repo has more code or more people. Today they would report almost nothing.

A guardrail that improves itself

In the three days since the first version of the map, 18 checks and settings have been added. Each of them went down the same road as any other change: a branch, a pull request, six checks, a merge. The guardrails map follows them card by card, and the improver proposes more every day. From what happened along the way come three lessons that hold outside this project too.

1 **A check that cannot fail lies.**

Verify that your guard really goes red when it should. The easiest way is to feed it a mistake, as with the unused variable and the syntax error in PR 96 and PR 98.

2 **Guards need tests, and changes to them need to be declared.**

The first hook had 36 failures in its first version. The list of guarded paths could be emptied until it was read from `main` as well.

3 **A rule in a document is mostly followed by an agent. A rule in CI is always followed.**

What a machine can verify belongs in a check. A document is for what takes judgement.

No slice of cheese is perfect. But when five of them lie on top of each other, and the holes are honestly written down and patched, a mistake needs a lot of luck to reach `main`.

Sources: the guardrails map (`docs/guardrails-map.html`), `docs/agents/git.md`, `docs/agents/critical-paths.md`, `docs/agents/guardrail-paths.md`, `.claude/settings.json`, `.claude/hooks/`, `.githooks/`, `.github/workflows/` and the bodies of pull requests 6, 81, 89, 96, 98, 99 and 100 in the siroroman/BreathApp repo. Status as of 1 November 2026.